# E03 — Scramble, then simplify

How often does Regina's greedy simplify() fail to undo random Pachner moves, and what does it take to recover?

regina_experiments
2026-10-30

---

## Summary

We took the minimal triangulations of 51 closed orientable manifolds, spanning every geometric class in Regina's census. We scrambled each with random Pachner moves, 9,180 times in all, and asked Regina to simplify them back. This is the first experiment run entirely through mtengine.

- **Everything comes back.** All 9,180 scrambled triangulations return to exactly the census size $n_0$, as the same manifold (checked by census lookup). Up to 160 random moves, scrambling never produced a triangulation that `simplifyExhaustive` at height 2 could not undo.
- **One greedy call fails 2.3% of the time.** That is, a single `simplify()` call stops above $n_0$ in 1,042 of 45,900 calls. The best of 5 calls still fails in 0.9% of scrambles. Of those 79 scrambles, exhaustive search at height 1 recovers 12 and height 2 recovers 67.
- **Random 4-4 moves make scrambles harder, more than longer walks do.** With 2-3 moves alone the failure rate climbs from 0.1% (5 moves) to about 1.5% (40 or more moves) and then levels off. Adding random 4-4 moves roughly triples it, from about 1% to about 3% on average over $k$.
- **Difficulty depends on the manifold, not the walk.** 30 of the 51 manifolds were ever hard, and a handful account for most failures. Lens spaces fail most often per call (4.1%). The hyperbolic manifolds fail 3.3% per call, but only 0.4% after 5 calls, because retrying helps them. The 3-sphere and the torus bundles never failed.
- **When greedy gets stuck, it stops close.** 77% of failed calls stop just one tetrahedron above $n_0$; the largest gap was 5.
- **mtengine reproduces E02 exactly.** E02's 1,494 hard non-minimal triangulations need the same exhaustive heights through mtengine's `simplify_triangulations`.
- **3-sphere recognition is fast.** All 50 scrambled 3-spheres (13–163 tetrahedra) were recognised in under 0.6 s each, including about 0.2 s to start Regina's process.

## Method

**Tools.** All Regina work runs through mtengine (submodule `engine/`, commit `768bbaa`), called from Python as `rexp.engine.mte`:

- `pachner_walk`: a seeded random walk of $k$ 2-3 moves, each optionally followed by `mix44` random 4-4 moves. One-tetrahedron triangulations admit no 2-3 move, so the walk uses a 1-4 move there; we added this to mtengine during this experiment, after the first run failed on $S^3$.

- `simplify_triangulations`: `trials` independent greedy `simplify()` calls (the function is randomised), then `simplifyExhaustive` at heights 1, 2, … until the target size is reached. Census lookup of the result.
- `triangulation_invariants` with `recognise=True`: 3-sphere recognition.

mtengine caches every result with its provenance (Regina 7.4.1, input hash, timing). The run directory records the engine commit.

**Part A: scramble and simplify.**

- *Manifolds.* $S^3$, plus 10 manifolds drawn at random (fixed seed) from each geometric class of the closed orientable census: lens spaces, Seifert fibred spaces, graph manifolds, torus bundles and hyperbolic manifolds. 51 in total, with minimal size $n_0$ from 1 to 10 tetrahedra.
- *Scrambles.* For each manifold, $k \in \{5, 10, 20, 40, 80, 160\}$ random 2-3 moves, with $\text{mix44} \in \{0, 1, 4\}$ random 4-4 moves after each, and 10 seeds each. That gives 9,180 scrambled triangulations of $n_0 + k$ tetrahedra.
- *Simplification.* 5 greedy calls each; if none reaches $n_0$, exhaustive search from the best result up to height 2.

**Part B: E02's hard set.** These are the 1,494 non-minimal triangulations (5–10 tetrahedra) that one `simplify()` call could not reduce in E02. Each was given 5 greedy calls, then exhaustive search up to height 3, with E02's target sizes.

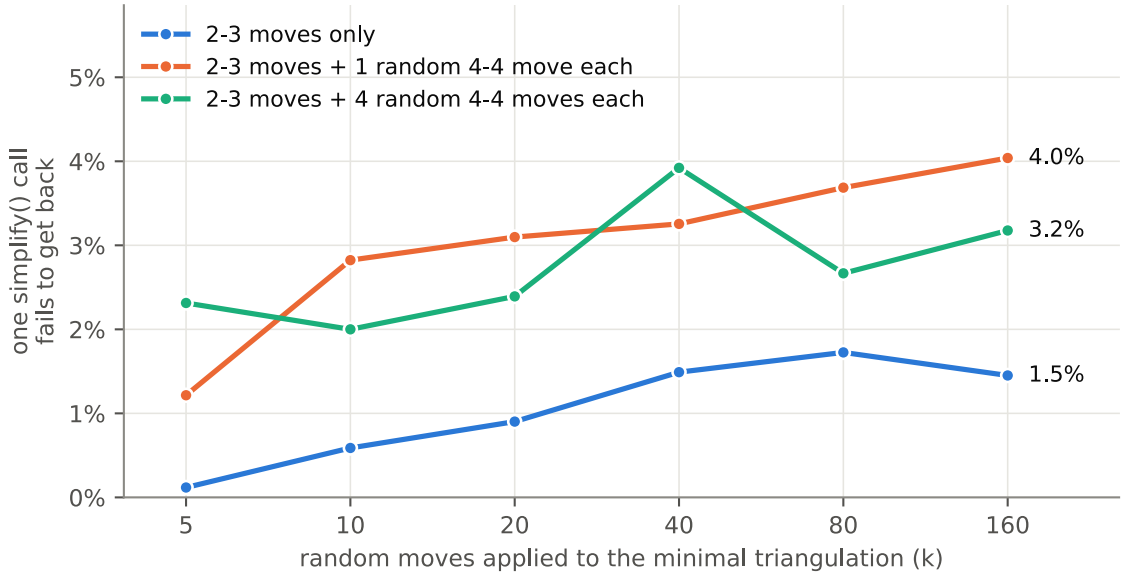
**Part C: 3-sphere recognition.** The 1-tetrahedron $S^3$, scrambled with $k \in \{10, 20, 40, 80, 160\}$ moves and 10 seeds each.

The whole run took 4 minutes on 4 cores.

## Results

### Part A: everything comes back, but not always in one call

| | |
|---|---|
| scrambled triangulations | 9,180 |
| returned to the census size $n_0$ | **9,180** |
| …identified as a different manifold | 0 |
| one `simplify()` call stays above $n_0$ | 1,042 of 45,900 calls (2.3%) |
| best of 5 calls stays above $n_0$ | 79 scrambles (0.9%) |
| …recovered by exhaustive height 1 / 2 | 12 / 67 |
| unresolved | 0 |




Probability that a single greedy `simplify()` call fails to return a scrambled triangulation to its minimal size, against the number of random 2-3 moves k, for three scrambling styles. Each point averages 2,550 calls (51 manifolds × 10 seeds × 5 calls).

| $k$ | **2-3 only** | **+ 1 random 4-4 move each** | **+ 4 random 4-4 moves each** |
|---:|---:|---:|---:|
| 5 | 0.1% | 1.2% | 2.3% |
| 10 | 0.6% | 2.8% | 2.0% |
| 20 | 0.9% | 3.1% | 2.4% |
| 40 | 1.5% | 3.3% | 3.9% |
| 80 | 1.7% | 3.7% | 2.7% |
| 160 | 1.5% | 4.0% | 3.2% |

**Reading the table.** Each cell rests on 2,550 calls, but the failures cluster on a few manifolds, so cell-to-cell differences of about 1 percentage point are within noise. Two effects are clear:

- **Mixing matters most.** Random 4-4 moves raise the failure rate from about 1% to about 3%. 4-4 moves preserve size, but they move the triangulation away from the "tree" of 2-3 moves that greedy 3-2 moves can simply retrace.
- **Walk length matters early, then stops mattering.** With 2-3 moves alone, failures grow from 5 to 40 moves and then level off. By 160 moves (170-tetrahedron triangulations), scrambles are no harder than at 40.

**By geometric class:**

| **class** | **scrambles** | **one call fails** | **best of 5 fails** |
|---|---:|---:|---:|
| $S^3$ | 180 | 0% | 0% |
| lens spaces | 1,800 | 4.1% | 1.4% |
| Seifert fibred | 1,800 | 1.8% | 1.1% |
| graph manifolds | 1,800 | 2.4% | 1.4% |

| class | scrambles | one call fails | best of 5 fails |
|---|---|---|---|
| torus bundles | 1,800 | 0% | 0% |
| hyperbolic | 1,800 | 3.3% | 0.4% |

The difference between the two failure columns says what kind of difficulty each class has:

- **Hyperbolic manifolds** fail often per call, but 5 calls almost always succeed. Their stuck states are escapable by the random moves `simplify()` already makes.
- **Lens, Seifert and graph manifolds** keep about 1–1.4% of failures after 5 calls. These are the ones that need exhaustive search.
- **Torus bundles and $S^3$** never failed in any of 1,980 scrambles and 9,900 calls.

**Hard manifolds.** 30 of the 51 manifolds had at least one failed call, but a few dominate:

- $L(55, 19)$: 80 of its 180 scrambles had a failed call;
- `Hyp_1.75712603`: 67; `Hyp_1.88541473`: 63;
- $L(47, 15)$: 57;
- the graph manifold SFS$[D : (2, 1)(3, 1)] \cup_m$ SFS$[D : (2, 1)(3, 2)]$: 38.

The 67 scrambles that needed height 2 come from just 9 manifolds. The largest share is 19 from the graph manifold SFS$[D : (2, 1)(3, 1)] \cup_m$ SFS$[D : (2, 1)(4, 1)]$, then 12 from SFS$[S^2 : (2, 1)(2, 1)(3, 1)(4, -7)]$ and 11 from $L(93, 16)$. `data/e03/stuck.tsv` lists every scramble with a failed call, with its seed, so each can be regenerated.

**How far above minimal greedy stops:**

| tetrahedra above $n_0$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| failed calls | 802 | 187 | 37 | 1 | 15 |

**Cost.** A `simplify()` call takes on average 0.2 ms after 5 moves (scrambles of 6–15 tetrahedra) and 6.8 ms after 160 moves (161–170 tetrahedra).

### Part B: E02's hard set, through mtengine

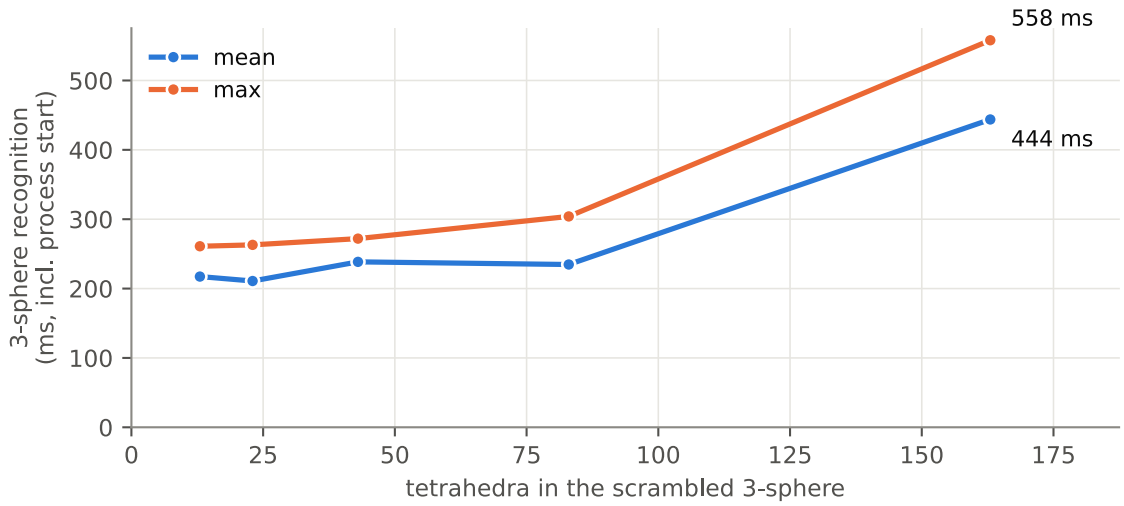
| E02 (one call): height needed | now (5 calls + exhaustive) | triangulations |
|---|---|---|
| 1 | solved by a greedy call | 407 |
| 1 | height 1 | 160 |
| 2 | height 2 | 926 |
| 3 | height 3 | 1 |

This matches E02 exactly. E02's retry experiment found that `simplify()` sometimes succeeds on 416 of the 567 height-1 cases, and 407 of them succeed within 5 calls here. Every height-2 and height-3 case still needs exhaustive search. The new mtengine tool and E02's direct Regina code agree on all 1,494 cases.

**Natural vs scrambled.** Part A's stuck states are typically one or two tetrahedra above minimal and need height at most 2, like E02's hard set. A random walk does not seem to create harder obstructions than those that already occur among small non-minimal

triangulations.

### Part C: 3-sphere recognition




Wall time of mtengine's `triangulation_invariants(..., recognise=True)` on scrambled 3-spheres. This includes about 0.2 s to start Regina's process.

All 50 scrambled 3-spheres were recognised. Beyond the roughly 0.2 s fixed cost, the time stays small up to 83 tetrahedra and is about 0.25 s extra at 163.

## Conclusions

1. **Random scrambling is a weak source of hard instances.** Even 160 random moves never produced a triangulation that height-2 exhaustive search could not undo. Benchmarks for simplification should prefer E02's naturally occurring hard set, or targeted constructions, over random walks.
2. **To get back to minimal reliably at this scale: run `simplify()` up to 5 times, then `simplifyExhaustive` at height 1 and then 2.** That recovered all 9,180 scrambles, with exhaustive search needed for 0.9%. This is what mtengine's `simplify_triangulations(trials=5, max_height=2, target_size=n0)` does.
3. **Mixing, not length, makes a scramble hard.** Random 4-4 moves triple the greedy failure rate; walk length stops mattering beyond about 40 moves.
4. **Difficulty is a property of the manifold.** Some lens spaces (such as $L(55, 19)$ and $L(93, 16)$), graph manifolds and hyperbolic manifolds are repeatedly hard. Torus bundles and $S^3$ never are. Why these manifolds? Their minimal triangulations may sit in sparse parts of the Pachner graph. This is a concrete question for E04.

## Next steps

- **E04:** for the hard manifolds above, measure how many minimal triangulations each has and how they connect in the Pachner graph, versus manifolds that are never hard.
- Construct hard instances deliberately, e.g. by layering on solid tori or using high-degree edges, rather than by random walks, and see whether height 3 is ever needed.

- Rerun with longer mixed walks (several 4-4 moves per step and occasional 3-2 moves), to see whether the 3–4% failure rate keeps rising.

## Reproduce

```
make engine                      # mtengine submodule + venv
make e03 ARGS="--moves 5 10 20 40 80 160 --mix44 0 1 4"   # ~4 min on 4 cores
make e03-export ARGS=results/e03_scramble_simplify/<run>
make writeups
```

Curated outputs are in `data/e03/`:

- `summary.json`: every table above;
- `report.txt`: the full analysis output;
- `stuck.tsv`: every scramble on which a greedy call failed, with its seed and isomorphism signature.